An Empirical Evaluation of a Pretrained Tabular Foundation Model on Small Numerical Tables: Kumo Tabular versus Untuned Classical Baselines

Anonymous authors
Paper under double-blind review

Abstract

Tabular foundation models predict labels for new rows by conditioning on labelled rows of a new dataset, without training on it. We report an independent evaluation of one such model, NVIDIA Kumo Tabular (smallest checkpoint, default settings), against seven untuned classical models—logistic or linear regression, random forests, extra trees, histogram gradient boosting, XGBoost, LightGBM and CatBoost—on 21 small, fully numerical datasets from OpenML-CC18 and OpenML-CTR23, using the official ten-fold splits, treating datasets as the unit of inference and following an analysis plan written after a one-dataset pipeline check and before the other results existed. For classification (15 datasets), Kumo Tabular had the lowest log loss on average (rank 1.07 of 8) and was better than each baseline on at least 14 datasets; all seven differences remained significant after Holm's correction and in two sensitivity analyses, and it also ranked first on ROC-AUC, indicating that the advantage is not only one of calibration. Its accuracy was nevertheless below the best baseline on three datasets. For regression (six datasets), it ranked first on RMSE but no pairwise difference was significant, and it was clearly worse than XGBoost on one dataset. Two case studies show that logistic regression matched it with 25 to 100 training rows and that linear regression was indistinguishable from it on a nearly linear regression task. Prediction cost on our hardware was comparable to large random forests but well above linear models. The results support Kumo Tabular as a strong default for small numerical tables, within the limits of the data regime, untuned baselines and a single checkpoint.

1 Introduction

Tree ensembles and regularized linear models have long been the practical default for tabular data, and extensive comparisons found gradient-boosted trees competitive with or better than deep networks on typical medium-sized tables (Shwartz-Ziv & Armon, 2022; Grinsztajn et al., 2022; Borisov et al., 2024). Tabular foundation models take a different approach: a transformer is pretrained once, mostly on synthetic tables, and then predicts the labels of new rows by conditioning on the labelled rows of a new dataset, without any training on it (Hollmann et al., 2023; 2025; Qu et al., 2025). NVIDIA Kumo Tabular (NVIDIA, 2026a; Qu et al., 2026) is a recently released model of this kind. Its developers report leading results on public benchmark suites, but we know of no independent evaluation.

Use of large language models and division of work. *Original study (author).* Before Claude's involvement, the author independently originated the research idea and hypothesis, developed the initial benchmark methodology, selected the evaluation metrics and baseline models, ran experiments on the two original datasets and performed an initial analysis of their results. *AI-assisted work.* Claude Opus 5.5 (Anthropic), used through Claude Code, re-implemented the two-dataset study from the author's specification and produced the two-dataset results reported here (Section 5.5, Appendix B), adding the repeated splits, the training-size study, the interval evaluation and the corrected significance tests. For the OpenML benchmark, the author chose to extend the study with OpenML datasets and chose the additional baseline libraries from options Claude proposed; Claude designed the extension (dataset inclusion criteria, baseline configurations, statistical analysis plan), wrote all code, ran the experiments, performed and interpreted the statistical analysis and checked the references. Claude drafted and revised the text, tables and figures, including the conclusions, following the author's instructions. ChatGPT (OpenAI) assisted the author with research planning, writing task specifications and review.

This paper reports an empirical evaluation of Kumo Tabular on small, fully numerical tables that is independent of the model's developers. It does not propose a new method. We ask:

RQ1 How does Kumo Tabular, used with default settings, compare with seven untuned classical models in predictive quality across an externally defined set of small datasets?

RQ2 Is any difference consistent across datasets, robust to dependence between related datasets, and present in ranking quality rather than only in probability calibration?

RQ3 Where does Kumo Tabular fail to outperform the classical models?

RQ4 What does it cost in runtime on modest hardware?

Our contributions are:

- A benchmark of Kumo Tabular against logistic or linear regression, random forests, extra trees, histogram gradient boosting, XGBoost, LightGBM and CatBoost on 21 OpenML datasets selected by fixed criteria, using the official OpenML splits (Section 4).
- A planned analysis with datasets as the unit of inference, a single primary metric per task, Holm-corrected pairwise tests, effect sizes and two sensitivity analyses, with the statistics recomputed by a second, separately written implementation (Section 5).
- Two detailed case studies, including a training-size study and an evaluation of predictive intervals, which illustrate how dataset-level conclusions can differ from the aggregate picture (Section 5.5).
- Code, raw per-fold results, logs and analysis scripts as anonymized supplementary material.

2 Related Work

Classical models for tabular data. Logistic regression (Cox, 1958) and least squares remain strong baselines for small, low-dimensional problems. Random forests (Breiman, 2001) and extremely randomized trees (Geurts et al., 2006) average decorrelated trees; gradient boosting (Friedman, 2001) fits trees sequentially to residuals, and its efficient implementations XGBoost (Chen & Guestrin, 2016), LightGBM (Ke et al., 2017) and CatBoost (Prokhorenkova et al., 2018) are the usual reference standard. We use the scikit-learn implementations (Pedregosa et al., 2011) where available.

Deep learning on tables. Comparisons of neural architectures such as TabNet (Arık & Pfister, 2021) and tabular transformers (Gorishniy et al., 2021) with tree ensembles found trees competitive or superior on many medium-sized datasets (Shwartz-Ziv & Armon, 2022; Borisov et al., 2024; Grinsztajn et al., 2022). Well-regularized multilayer perceptrons can close the gap (Kadra et al., 2021), and the best family varies with dataset properties (McElfresh et al., 2023).

In-context learning and tabular foundation models. In-context learning, in which a pretrained model solves a task from examples in its input without weight updates, was popularized by large language models (Brown et al., 2020) built on transformers (Vaswani et al., 2017), and emerges for simple function classes under suitable training distributions (Garg et al., 2022). Language models have also been applied to serialized tables (Hegselmann et al., 2023). Prior-data fitted networks (Müller et al., 2022) are trained on datasets sampled from a prior so that a forward pass approximates the posterior predictive distribution. TabPFN (Hollmann et al., 2023) applied this to small classification tables, and its successor (Hollmann et al., 2025) extended it to regression and larger inputs. TabICL (Qu et al., 2025) compresses rows with column- and row-wise attention before in-context learning; set attention with inducing points (Lee et al., 2019) is a common component.

Kumo Tabular. Kumo Tabular (NVIDIA, 2026a; Qu et al., 2026) follows this line. Its developers describe column, row and in-context attention "as introduced in TabICL and TabPFN", pretraining on synthetic tables sampled from structural causal models, and three sizes with 28M to 215M parameters; they report leading results on several benchmark suites. Those claims are the developers' own. We are not aware of an independent evaluation of Kumo Tabular.

Benchmarking methodology. OpenML (Vanschoren et al., 2014) provides datasets with fixed evaluation splits; OpenML-CC18 (Bischl et al., 2021) and OpenML-CTR23 (Fischer et al., 2023) are curated classification and regression suites. For comparisons across datasets we follow Demšar (2006): a Friedman test (Friedman, 1937) over all methods, with pairwise Wilcoxon signed-rank tests (Wilcoxon, 1945) and Holm's correction (Holm, 1979). For comparisons within one dataset over resampled splits, Dietterich (1998) showed that naive paired tests are anti-conservative, and Nadeau & Bengio (2003) proposed the corrected resampled t-test we use for the secondary within-dataset analysis.

3 Background: Training Versus In-Context Learning

Let $\mathcal{D}_{\mathrm{tr}} = \{(\boldsymbol{x}_i, y_i)\}_{i=1}^{n}$ be a labelled training set and $\boldsymbol{x}_q$ a query row. A classical model fits parameters to each dataset,

$$\hat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} \; \frac{1}{n} \sum_{i=1}^{n} \ell\big(f_{\boldsymbol{\theta}}(\boldsymbol{x}_i), y_i\big) + \lambda\, \Omega(\boldsymbol{\theta}), \tag{1}$$

and predicts $\hat{y}_q = f_{\hat{\boldsymbol{\theta}}}(\boldsymbol{x}_q)$. A tabular foundation model instead has parameters $\boldsymbol{\phi}$ fitted once, before the target dataset is seen, by minimizing the expected predictive loss over datasets drawn from a generator,

$$\boldsymbol{\phi}^{\star} = \arg\min_{\boldsymbol{\phi}} \; \mathbb{E}_{\mathcal{D}\sim p(\mathcal{D})}\Big[-\log q_{\boldsymbol{\phi}}\big(y_q \mid \boldsymbol{x}_q, \mathcal{D}_{\mathrm{ctx}}\big)\Big], \tag{2}$$

so that $q_{\boldsymbol{\phi}}$ approximates the posterior predictive distribution implied by the generator (Müller et al., 2022). On a new dataset, $\boldsymbol{\phi}^{\star}$ is frozen and the training set is supplied as context: $\hat{p}(y_q \mid \boldsymbol{x}_q) = q_{\boldsymbol{\phi}^{\star}}(y_q \mid \boldsymbol{x}_q, \mathcal{D}_{\mathrm{tr}})$. No gradient step is taken on the target data; every prediction call processes the context again unless context representations are cached.

Kumo Tabular as used here. From the developers' description (Qu et al., 2026) and the source code of `structured-data-models` 0.2.1 (NVIDIA, 2026b): cell values are embedded with learned Fourier features; a row encoder alternates column-wise induced self-attention (each row attends only to context rows) with row-wise attention and summarizes each row with four readout tokens; a final transformer performs in-context learning, with query rows attending to context rows only. The classification head supports up to ten classes and is wrapped in an error-correcting output code scheme (Dietterich & Bakiri, 1995); the regression head outputs 999 quantiles (Koenker & Bassett, 1978). A built-in recipe forms an ensemble of E preprocessed views (standardization followed by one of four transformations, column shuffling and, for classification, cyclic label shifts); class scores are averaged and normalized with a softmax, and regression quantiles are combined with a 20% trimmed mean. We use the `small` checkpoints (27.5M parameters for classification and 28.5M for regression, counted from the loaded models) with $E = 8$ and the median quantile as the regression point prediction.

4 Experimental Design

The study has two parts. The *main benchmark* (Sections 4.1–4.5) compares Kumo Tabular with seven classical models on 21 public datasets, using the dataset as the unit of inference. The *case studies* (Section 4.7) are a re-implementation, with additional analyses, of the author's earlier two-dataset study, and are reported as secondary, dataset-specific evidence. All case-study numbers come from this re-implementation; the outputs of the earlier experiments are not available, and we make no claim that they are reproduced here. The dataset selection criteria, splits, models and computed metrics were fixed before the benchmark was run. The analysis plan—primary metrics, statistical tests and sensitivity analyses—was written down and

Table 1: Datasets of the OpenML benchmark, selected from OpenML-CC18 and OpenML-CTR23 by fixed criteria (150–2,000 rows, at most 60 features, numerical features only, no missing values, at most 10 classes). `forest_fires` (OpenML 44962) met the criteria according to its OpenML metadata but contains two text columns and was excluded.

Classification					Regression			
ID	Name	Rows	Feat.	Cls.	ID	Name	Rows	Feat.
11	balance-scale	625	4	3	44957	airfoil_self_noise	1503	5
18	mfeat-morphological	2000	6	10	44959	concrete_compressive_strength	1030	8
22	mfeat-zernike	2000	47	10	44960	energy_efficiency	768	8
37	diabetes	768	8	2	44970	QSAR_fish_toxicity	908	6
54	vehicle	846	18	4	44972	red_wine	1599	11
1049	pc4	1458	37	2	44994	cars	804	17
1050	pc3	1563	37	2				
1063	kc2	522	21	2				
1068	pc1	1109	21	2				
1462	banknote-authentication	1372	4	2				
1464	blood-transfusion-service-center	748	4	2				
1494	qsar-biodeg	1055	41	2				
1510	wdbc	569	30	2				
40982	steel-plates-fault	1941	27	7				
40994	climate-model-simulation-crashes	540	20	2				

committed to a local version-control repository after a pipeline check on one dataset (`wdbc`), whose complete results for all eight models we had seen, and before results for any other dataset existed (supplementary material). The plan is therefore not a formal preregistration: it has no external timestamp, and it was written with knowledge of one of the 21 datasets. One planned sensitivity analysis excludes `wdbc`.

4.1 Datasets

We took all datasets from OpenML-CC18 (Bischl et al., 2021) and OpenML-CTR23 (Fischer et al., 2023) that have 150 to 2,000 rows, at most 60 features, only numerical features, no missing values and, for classification, at most ten classes (the limit of the Kumo Tabular classification head). The filters selected 15 classification and 7 regression datasets. One of them, `forest_fires`, contained two text columns although its OpenML metadata listed none; it was excluded under the same criterion when loading revealed this, before any of its results existed. This leaves 15 classification and 6 regression datasets (Table 1). The restrictions define the regime we study—small, fully numerical tables—and avoid preprocessing choices (encoding, imputation, subsampling) that would confound the comparison. They also limit the scope of our conclusions (Section 8). One dataset, `wdbc`, is the Breast Cancer Wisconsin data also used in the case studies.

4.2 Splits and seeds

We use the official OpenML task splits: 10-fold cross-validation, stratified for classification, first repetition for every task. The fold index (0–9) seeds every model and the random number generator of Kumo Tabular, whose predictions are deterministic given the seed.

4.3 Models

All models receive the same folds and no hyperparameter tuning; the comparison is between out-of-the-box methods.

- **Kumo Tabular** `small`, 8 ensemble members, training fold as context, test fold as query, float32 inference (float16 autocast is disabled on GPUs older than compute capability 7.0).
- **Logistic regression** on standardized inputs (`max_iter`=5000) for classification; **ordinary least squares** for regression.

- **Random forest** and **extra trees**, 500 trees each.
- **Histogram gradient boosting** (scikit-learn), **XGBoost**, **LightGBM** and **CatBoost**, library defaults.

4.4 Metrics

The primary metrics, fixed in advance, are the log loss for classification, a proper scoring rule for probabilistic predictions (Gneiting & Raftery, 2007), and the root mean squared error (RMSE) for regression. Secondary metrics are accuracy, macro-averaged F1 and one-vs-rest macro-averaged ROC-AUC (Fawcett, 2006) for classification, and mean absolute error and R^2 for regression. Each metric is computed per fold and averaged over the ten folds of a dataset. For models that do not output a probability for a class absent from a training fold, that probability is zero.

4.5 Statistical analysis

Folds of one dataset share most of their training rows and are not independent, so we use the per-dataset mean as the unit of analysis and treat datasets as independent replicates (Demšar, 2006). For each task and metric we apply a Friedman test (Friedman, 1937) to the ranks of all eight models, and compare Kumo Tabular with each baseline by a two-sided Wilcoxon signed-rank test on the per-dataset means, with Holm's correction (Holm, 1979) across the seven baselines. Only the primary metrics support confirmatory statements. We report wins, ties and losses and the median relative advantage as effect sizes. With six regression datasets the smallest attainable two-sided Wilcoxon p-value is $2/2^6 \approx 0.031$, which exceeds 0.05 after Holm's correction for seven comparisons: the regression results are descriptive by design. The plan includes two sensitivity analyses: excluding `wdbc`, which overlaps with the case studies (and is the dataset seen before the plan was written), and averaging related datasets into single units (the NASA software-defect datasets `pc1`, `pc3`, `pc4`, `kc2`; the two `mfeat` datasets) to reduce dependence between datasets.

4.6 Runtime

We record wall-clock fit and prediction time per fold. Kumo Tabular has no fitting stage; all of its cost is prediction. It ran on an NVIDIA GeForce GTX 980M (8 GB) and the baselines on an Intel Core i7-6700HQ CPU, with one untimed warm-up call per dataset for Kumo Tabular. Runtimes therefore compare complete pipelines on the available hardware, not algorithms, and are reported descriptively.

4.7 Case studies

The case studies use the scikit-learn copies of Breast Cancer Wisconsin (569 rows, 30 features, malignant as positive class) and Diabetes (442 rows, 10 features, original units) (Street et al., 1993; Efron et al., 2004), five 80/20 splits (seeds 42–46, stratified for classification), the first three baselines above, threshold metrics and RMSE, a training-size study (25 to all training rows; seeds 42–44), the coverage of Kumo Tabular's nominal 90% quantile interval, and a within-dataset paired analysis with the Nadeau–Bengio corrected resampled t-test (Nadeau & Bengio, 2003) and Holm's correction. Because the splits of one dataset are dependent and only five, this analysis is descriptive.

4.8 Leakage controls and verification

Kumo Tabular receives the training fold with labels and the test fold without its target column; unit tests check that no test-row label reaches the model. OpenML splits are checked to be disjoint within each fold and to cover every row exactly once. Preprocessing (standardization for logistic regression; Kumo Tabular's internal recipe) is fitted on the training fold only. All case-study results were re-run with bit-identical metrics, and the case-study notebooks (Anonymous, 2026) were executed in a separate cloud environment (NVIDIA T4, PyTorch 2.11, float16 inference), where results for Kumo Tabular, logistic and linear regression and random forests matched at the reported precision while histogram gradient boosting differed slightly, presumably because of a different scikit-learn version.

Table 2: Kumo Tabular versus each baseline on the primary metric, with datasets as the unit of analysis. W/T/L: datasets on which Kumo is better/equal/worse. Relative advantage: median over datasets of the baseline's loss minus Kumo's loss, divided by the baseline's loss (positive favours Kumo). p: two-sided Wilcoxon signed-rank test; Holm: adjusted across the seven baselines of a task. Avg. rank: average rank among all eight models (1 = best).

Task (metric)	Baseline	W/T/L	Rel. adv.	p	Holm p	Avg. rank
Classification (log loss) $n = 15$	Kumo Tabular	–	–	–	–	1.07
	CatBoost	15/0/0	+12.8%	<0.001	<0.001	2.47
	Logistic reg.	14/0/1	+20.5%	<0.001	<0.001	3.73
	Random forest	15/0/0	+40.2%	<0.001	<0.001	5.00
	XGBoost	15/0/0	+33.0%	<0.001	<0.001	5.07
	Extra trees	15/0/0	+54.5%	<0.001	<0.001	6.00
	HistGB	15/0/0	+42.8%	<0.001	<0.001	6.33
	LightGBM	15/0/0	+40.8%	<0.001	<0.001	6.33
Regression (RMSE) $n = 6$	Kumo Tabular	–	–	–	–	1.67
	CatBoost	5/0/1	+2.7%	0.156	0.312	2.67
	Extra trees	5/0/1	+10.4%	0.094	0.281	3.50
	XGBoost	5/0/1	+12.8%	0.156	0.312	4.17
	Random forest	5/0/1	+9.6%	0.062	0.250	4.33
	HistGB	6/0/0	+15.8%	0.031	0.219	5.33
	LightGBM	6/0/0	+15.9%	0.031	0.219	6.33
	Linear reg.	6/0/0	+48.7%	0.031	0.219	8.00

5 Results

All 1,680 model runs (21 datasets × 10 folds × 8 models) completed without failure. The analyses below follow the written analysis plan (Section 4); the statistics were recomputed from the raw per-fold results by a second, separately written implementation (exact Wilcoxon enumeration, hand-coded Friedman and Holm procedures) with identical results. Both implementations were written with the same AI assistant (see the note on page 1), so this check guards against coding errors, not against a shared misunderstanding.

5.1 Classification

On the primary metric, log loss, Kumo Tabular had the best average rank among the eight models (1.07; Friedman $p < 0.001$; Table 3, Figure 1). It had a lower log loss than CatBoost, random forests, extra trees, histogram gradient boosting, XGBoost and LightGBM on all 15 datasets, and than logistic regression on 14 of 15 (Table 2). All seven differences remain significant after Holm's correction (adjusted $p < 0.001$). The median relative reduction in log loss ranged from 12.8% against CatBoost to 54.5% against extra trees. Against the strongest baseline on each individual dataset, Kumo Tabular had the lower log loss on 14 of 15 datasets; on `climate-model-simulation-crashes` logistic regression was marginally better (0.1311 versus 0.1313; Figure 2).

Because log loss rewards calibrated probabilities, and tree ensembles are often poorly calibrated, a log-loss advantage could reflect calibration alone. The secondary metrics speak against this explanation. Kumo Tabular also had the best average rank for one-vs-rest ROC-AUC (1.33), which depends only on the ordering of predictions, and for accuracy (1.77) and macro-F1 (2.20). These secondary comparisons are descriptive, but each of the seven pairwise comparisons on each secondary metric favoured Kumo Tabular on at least 11 of 15 datasets. Its advantage in accuracy is not uniform: relative to the best baseline on each dataset it was lower on `diabetes` (−1.8 percentage points), `climate-model-simulation-crashes` (−1.1) and `kc2` (−0.2), equal on `pc3`, and largest on `vehicle` (+11.5) and `balance-scale` (+10.4).

Table 3: Average rank of each model per metric (1 = best) and Friedman test p-value. Primary metrics in the first column of each task; the others are descriptive.

	Classification ($n = 15$)				Regression ($n = 6$)		
Model	log loss	accuracy	ROC-AUC	macro-F1	RMSE	MAE	R^2
Kumo Tabular	1.07	1.77	1.33	2.20	1.67	1.33	1.67
Logistic reg.	3.73	3.93	4.40	4.80	–	–	–
Linear reg.	–	–	–	–	8.00	8.00	8.00
Random forest	5.00	6.20	5.27	6.07	4.33	4.67	4.33
Extra trees	6.00	5.27	5.60	5.80	3.50	3.17	3.50
HistGB	6.33	5.23	5.40	4.60	5.33	5.67	5.50
XGBoost	5.07	4.60	5.13	3.73	4.17	3.67	4.33
LightGBM	6.33	5.23	5.93	4.60	6.33	6.67	6.17
CatBoost	2.47	3.77	2.93	4.20	2.67	2.83	2.50
Friedman p	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001

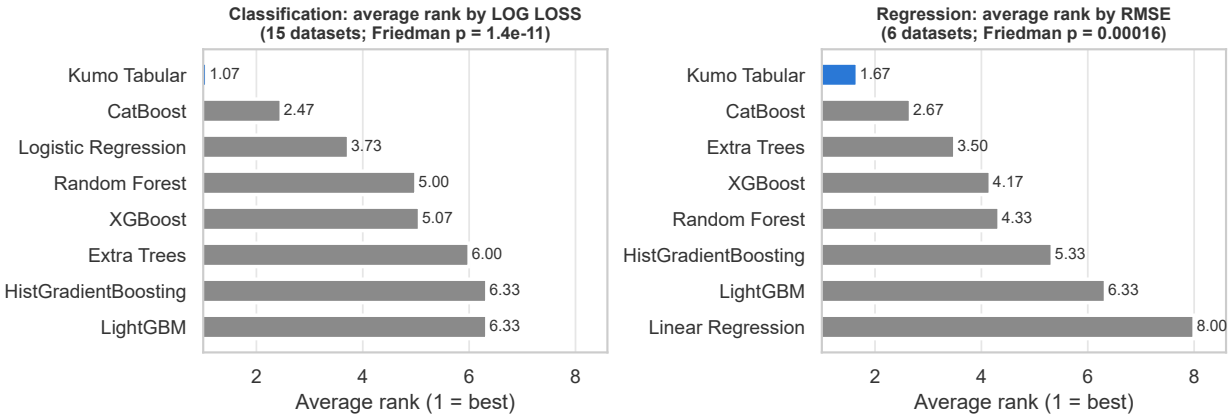


Figure 1: Average rank of each model on the primary metric across datasets (1 = best). Kumo Tabular in blue.

5.2 Regression

On RMSE, Kumo Tabular had the best average rank (1.67; Friedman $p < 0.001$), followed by CatBoost (2.67) and extra trees (3.50); ordinary least squares ranked last on every dataset. Kumo Tabular had a lower RMSE than histogram gradient boosting, LightGBM and least squares on all six datasets and than the other four baselines on five of six. None of these differences is significant after Holm's correction (smallest adjusted $p = 0.22$), as anticipated by the design: with six datasets these comparisons cannot reach significance, and we treat them as descriptive. Relative to the best baseline on each dataset, Kumo Tabular had a lower RMSE on four of six datasets, with the largest reduction on `airfoil_self_noise` (44%). It was clearly worse on `energy_efficiency`, where its RMSE was 44% higher than XGBoost's, and slightly worse on `red_wine` (5% higher than extra trees). The secondary metrics MAE and R^2 gave the same ordering of Kumo Tabular first (average ranks 1.33 and 1.67).

5.3 Sensitivity analyses

The classification conclusions did not change in either planned sensitivity analysis (Table 4). Without `wdbc`, Kumo Tabular's average rank was 1.07 and it was significantly better than all seven baselines after Holm's correction; with related datasets averaged into single units (11 units), its average rank was 1.09 and all seven comparisons remained significant (adjusted $p = 0.007$). The regression results were unchanged because no regression dataset was affected.

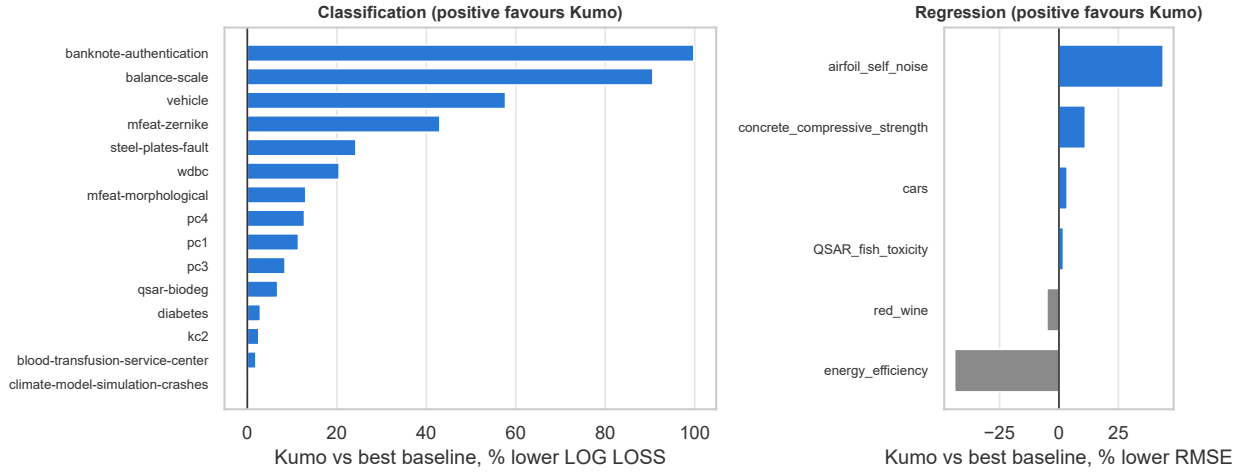


Figure 2: Per-dataset difference between Kumo Tabular and the best of the seven baselines on that dataset, as a percentage of the baseline's loss (log loss for classification, RMSE for regression). Positive values favour Kumo Tabular. Comparing against the per-dataset best baseline is stricter than any single fixed baseline.

Table 4: Planned sensitivity analyses on the primary metric: Kumo Tabular's average rank, the number of baselines it beats with Holm-adjusted $p < 0.05$, and its W/T/L against the strongest baseline (lowest average rank).

Task	Analysis	n	Kumo avg. rank	Sig. wins (of 7)	W/T/L vs strongest
Classification	All datasets	15	1.07	7	15/0/0 (CatBoost)
	Excluding wdbc	14	1.07	7	14/0/0 (CatBoost)
	Families collapsed	11	1.09	7	11/0/0 (CatBoost)
Regression	All datasets	6	1.67	0	5/0/1 (CatBoost)
	Excluding wdbc	6	1.67	0	5/0/1 (CatBoost)
	Families collapsed	6	1.67	0	5/0/1 (CatBoost)

5.4 Runtime

Table 5 and Figure 3 report fit-plus-prediction time per fold. On this hardware the median time per fold of Kumo Tabular on the GPU (1.27 s for classification, 1.14 s for regression) was comparable to 500-tree random forests and extra trees on the CPU and lower than CatBoost with its default 1,000 iterations. It was about 7 (classification) and 9 to 11 (regression) times the median time of XGBoost and LightGBM, and about 48 times that of logistic regression and 300 times that of least squares. The time per fold depended strongly on the dataset: on the 10-class `mfeat-zernike` dataset the ten folds took 1,113 s for histogram gradient boosting, 841 s for CatBoost and 650 s for Kumo Tabular, against 21 s or less for XGBoost, LightGBM, extra trees and logistic regression. These measurements are hardware-specific (Section 8).

5.5 Case studies

The two-dataset case studies (Appendix B) are consistent with the benchmark but add detail. On breast cancer data Kumo Tabular had the best mean on every threshold metric (accuracy 0.989 versus 0.977 for logistic regression) and higher accuracy than logistic regression on all five splits, but after the overlap correction and Holm's adjustment no within-dataset comparison was significant (smallest adjusted $p = 0.081$). On diabetes data it was indistinguishable from linear regression (R^2 0.503 versus 0.496). In the training-size study it was much more robust than least squares with 25 training rows (R^2 0.429 versus 0.075) but no better than logistic regression for classification at 25 to 100 rows. Its nominal 90% quantile interval covered 93.3% of diabetes test targets.

Table 5: Fit plus prediction time per fold in seconds: median [interquartile range] over all folds of all datasets. Kumo Tabular ran on an NVIDIA GTX 980M GPU (float32); all other models on an Intel Core i7-6700HQ CPU. Times are hardware-dependent and not comparable as algorithmic costs.

Model	Classification	Regression
Kumo Tabular	1.275 [0.896, 3.053]	1.142 [0.929, 1.471]
Logistic reg.	0.027 [0.015, 0.053]	–
Linear reg.	–	0.004 [0.003, 0.005]
Random forest	1.392 [1.284, 1.542]	1.077 [1.038, 1.165]
Extra trees	1.087 [1.001, 1.242]	0.753 [0.737, 0.789]
HistGB	0.487 [0.323, 1.474]	0.379 [0.341, 0.403]
XGBoost	0.181 [0.114, 0.283]	0.125 [0.110, 0.152]
LightGBM	0.168 [0.094, 0.318]	0.103 [0.092, 0.117]
CatBoost	4.564 [3.251, 5.824]	1.728 [1.348, 2.018]

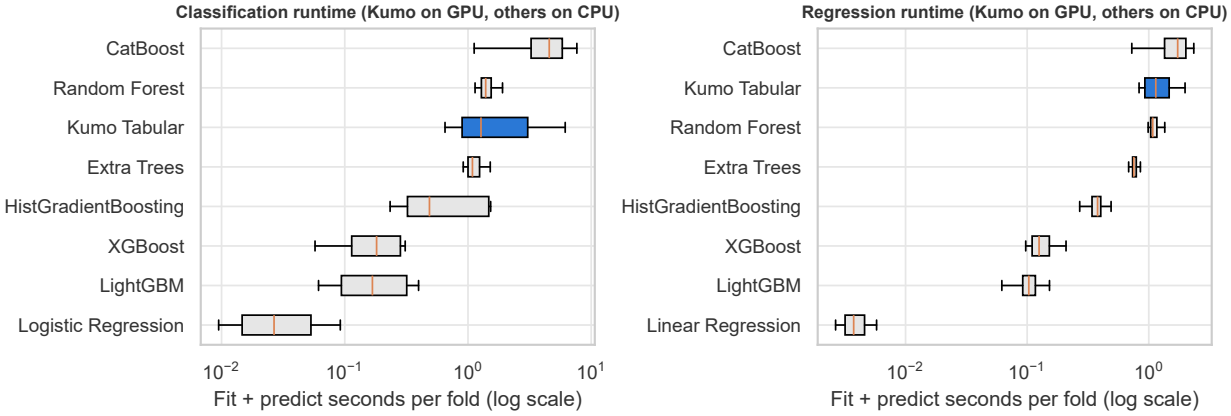


Figure 3: Fit plus prediction time per fold (log scale) over all folds of all datasets. Kumo Tabular ran on a GPU, the other models on a CPU.

6 Discussion

What the evidence establishes. On 15 small, fully numerical classification datasets, the `small` Kumo Tabular checkpoint with default settings produced better probabilistic predictions than each of seven untuned classical models, consistently across datasets, with differences that survive multiplicity correction and both sensitivity analyses (RQ1, RQ2). The advantage also appears in ROC-AUC, so it is not explained by calibration alone. On six regression datasets it ranked first on average, but the evidence is too limited for a statistical claim.

Where it does not help. The aggregate hides dataset-level exceptions (RQ3). Kumo Tabular's accuracy was below the best baseline on three of 15 classification datasets, and its RMSE was 44% higher than XGBoost's on `energy_efficiency`. In the case studies, logistic regression matched it for classification with 25 to 100 training rows, and on the diabetes data linear regression was statistically indistinguishable from it. The largest accuracy gains occurred on two multiclass datasets (`vehicle`, `balance-scale`); on several binary tasks the differences in accuracy were a fraction of a percentage point.

Interpreting the comparison. The baselines were not tuned, whereas Kumo Tabular's defaults result from extensive pretraining. Our comparison therefore answers a practical question—what a practitioner obtains without model selection on small numerical tables—and not which model class is better when each is tuned. Regularized or tuned gradient boosting, and a strong linear model for nearly linear targets, remain relevant alternatives.

Cost. Prediction with Kumo Tabular processes the full context at every call. On our hardware its time per fold was comparable to large random forests and smaller than CatBoost's, but well above linear models, XGBoost and LightGBM, and it requires a GPU for reasonable speed (RQ4).

7 Conclusion

In an independent evaluation on 21 small numerical OpenML datasets, the smallest Kumo Tabular checkpoint outperformed seven untuned classical baselines on classification log loss on nearly every dataset, and ranked first on average for regression without statistical support. The results justify Kumo Tabular as a strong default for small numerical tables, while the dataset-level exceptions, the restricted data regime and the untuned baselines limit how far the conclusion extends.

8 Limitations and Threats to Validity

Scope of the datasets. All datasets are small (at most 2,000 rows), fully numerical and complete. Our conclusions do not extend to larger tables, categorical or text features, or missing values—settings that tabular foundation models are designed to handle and in which their relative performance may differ. The datasets come from two curated suites; curation and the selection filters shape which problems are represented. The filters relied on OpenML metadata, which was wrong for `forest_fires`: it was excluded at load time because of undeclared text columns, after results for 17 other datasets existed. The exclusion follows the selection criterion and did not depend on any result, but it reduced the regression suite from seven to six datasets, and we did not evaluate the dataset with its text columns encoded or removed. Six regression datasets support descriptive conclusions only.

Dependence between datasets. Some datasets share a source (four NASA software-defect datasets; two `mfeat` feature sets of the same digits), which violates the independence assumed by the Friedman and Wilcoxon tests. The planned family-collapsed analysis addresses this only partly.

Untuned baselines and a single checkpoint. No model was tuned. Tuned gradient boosting or regularized linear models could narrow or reverse differences, and library defaults are not designed for very small data. Only the `small` Kumo Tabular checkpoint was evaluated, with float32 inference on an older GPU; the developers' claims concern larger checkpoints as well.

Pretraining overlap. The developers state that Kumo Tabular was pretrained on synthetic data only (Qu et al., 2026). We cannot verify this; if any benchmark dataset or close variant was seen in pretraining, Kumo Tabular's performance on it would be overstated.

Statistical power and multiplicity. With 15 classification datasets the cross-dataset tests can detect only consistent differences; absence of significance is not evidence of equivalence. Secondary metrics and sensitivity analyses are not adjusted for multiplicity and are descriptive. Within-dataset comparisons in the case studies rest on five dependent splits.

Timing of the analysis plan. The analysis plan was written after the complete results on `wdbc` had been seen, on which Kumo Tabular was best on all quality metrics, and it is timestamped only in a local repository. Choices made in the plan, such as the primary metrics, could in principle have been influenced by that one dataset. The analysis without `wdbc` gives the same conclusions (Table 4), and the secondary metrics are reported in full.

Runtime. Runtimes come from one laptop with a GPU for Kumo Tabular and a CPU for all other models, and timings of a few milliseconds vary between runs. They indicate orders of magnitude on this hardware, not algorithmic costs.

Construct validity. Accuracy and macro-F1 use the arg-max decision rule; in applications the decision threshold would be chosen for a target cost. The medical case-study datasets are used as benchmarks only; no conclusion about clinical use is drawn.

Reproducibility Statement

The anonymized supplementary material (Anonymous, 2026) contains all code, the dataset selection script and its output, raw per-fold and per-split results, run logs, environment records, the analysis plan and the analysis and validation scripts. The datasets and splits are public on OpenML; the model weights are public on Hugging Face. All 45 unit tests pass, including tests for data leakage and prediction shapes. The OpenML benchmark ran for about 1.5 hours on the hardware in Appendix C.

Broader Impact Statement

This is a benchmark study of an existing, publicly released model on public datasets. We do not foresee a significant risk of harm. Two case-study datasets are medical; they are used only as benchmarks, and our results must not be read as evidence about clinical use.

References

Anonymous. Code, notebooks and raw results for this submission. Anonymized supplementary material, 2026.

S. Ö. Arık and T. Pfister. TabNet: Attentive interpretable tabular learning. In *Proc. AAAI Conf. Artif. Intell.*, volume 35, pp. 6679–6687, 2021. doi: 10.1609/aaai.v35i8.16826.

B. Bischl, G. Casalicchio, M. Feurer, P. Gijsbers, F. Hutter, M. Lang, R. G. Mantovani, J. N. van Rijn, and J. Vanschoren. OpenML benchmarking suites. In *Proc. Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021. arXiv:1708.03731.

V. Borisov, T. Leemann, K. Seßler, J. Haug, M. Pawelczyk, and G. Kasneci. Deep neural networks and tabular data: A survey. *IEEE Trans. Neural Netw. Learn. Syst.*, 35(6):7499–7519, 2024. doi: 10.1109/TNNLS.2022.3229161.

L. Breiman. Random forests. *Mach. Learn.*, 45(1):5–32, 2001. doi: 10.1023/A:1010933404324.

T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, et al. Language models are few-shot learners. In *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2020. arXiv:2005.14165.

T. Chen and C. Guestrin. XGBoost: A scalable tree boosting system. In *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, pp. 785–794, 2016. doi: 10.1145/2939672.2939785.

D. R. Cox. The regression analysis of binary sequences. *J. Roy. Statist. Soc. Ser. B*, 20(2):215–232, 1958. doi: 10.1111/j.2517-6161.1958.tb00292.x.

J. Demšar. Statistical comparisons of classifiers over multiple data sets. *J. Mach. Learn. Res.*, 7(1):1–30, 2006.

T. G. Dietterich. Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Comput.*, 10(7):1895–1923, 1998. doi: 10.1162/089976698300017197.

T. G. Dietterich and G. Bakiri. Solving multiclass learning problems via error-correcting output codes. *J. Artif. Intell. Res.*, 2:263–286, 1995. doi: 10.1613/jair.105.

B. Efron, T. Hastie, I. Johnstone, and R. Tibshirani. Least angle regression. *Ann. Statist.*, 32(2):407–499, 2004. doi: 10.1214/009053604000000067.

T. Fawcett. An introduction to ROC analysis. *Pattern Recognit. Lett.*, 27(8):861–874, 2006. doi: 10.1016/j.patrec.2005.10.010.

S. F. Fischer, L. Harutyunyan, M. Feurer, and B. Bischl. OpenML-CTR23 – a curated tabular regression benchmarking suite. In *AutoML Conference 2023 (Workshop)*, 2023. URL `https://openreview.net/forum?id=HebAOoMm94`.

J. H. Friedman. Greedy function approximation: A gradient boosting machine. *Ann. Statist.*, 29(5):1189–1232, 2001. doi: 10.1214/aos/1013203451.

M. Friedman. The use of ranks to avoid the assumption of normality implicit in the analysis of variance. *J. Amer. Statist. Assoc.*, 32(200):675–701, 1937. doi: 10.1080/01621459.1937.10503522.

S. Garg, D. Tsipras, P. Liang, and G. Valiant. What can transformers learn in-context? A case study of simple function classes. In *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2022. arXiv:2208.01066.

P. Geurts, D. Ernst, and L. Wehenkel. Extremely randomized trees. *Mach. Learn.*, 63(1):3–42, 2006. doi: 10.1007/s10994-006-6226-1.

T. Gneiting and A. E. Raftery. Strictly proper scoring rules, prediction, and estimation. *J. Amer. Statist. Assoc.*, 102(477):359–378, 2007. doi: 10.1198/016214506000001437.

Y. Gorishniy, I. Rubachev, V. Khrulkov, and A. Babenko. Revisiting deep learning models for tabular data. In *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2021. arXiv:2106.11959.

L. Grinsztajn, E. Oyallon, and G. Varoquaux. Why do tree-based models still outperform deep learning on typical tabular data? In *Adv. Neural Inf. Process. Syst. (NeurIPS), Datasets and Benchmarks Track*, 2022. arXiv:2207.08815.

S. Hegselmann, A. Buendia, H. Lang, M. Agrawal, X. Jiang, and D. Sontag. TabLLM: Few-shot classification of tabular data with large language models. In *Proc. Int. Conf. Artif. Intell. Statist. (AISTATS)*, volume 206 of *Proc. Mach. Learn. Res.*, pp. 5549–5581, 2023.

N. Hollmann, S. Müller, K. Eggensperger, and F. Hutter. TabPFN: A transformer that solves small tabular classification problems in a second. In *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023. arXiv:2207.01848.

N. Hollmann, S. Müller, L. Purucker, A. Krishnakumar, M. Körfer, S. B. Hoo, R. T. Schirrmeister, and F. Hutter. Accurate predictions on small data with a tabular foundation model. *Nature*, 637(8045):319–326, 2025. doi: 10.1038/s41586-024-08328-6.

S. Holm. A simple sequentially rejective multiple test procedure. *Scand. J. Statist.*, 6:65–70, 1979.

A. Kadra, M. Lindauer, F. Hutter, and J. Grabocka. Well-tuned simple nets excel on tabular datasets. In *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2021. arXiv:2106.11189.

G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017.

R. Koenker and G. Bassett. Regression quantiles. *Econometrica*, 46(1):33–50, 1978. doi: 10.2307/1913643.

J. Lee, Y. Lee, J. Kim, A. R. Kosiorek, S. Choi, and Y. W. Teh. Set transformer: A framework for attention-based permutation-invariant neural networks. In *Proc. Int. Conf. Mach. Learn. (ICML)*, 2019. arXiv:1810.00825.

D. McElfresh, S. Khandagale, J. Valverde, V. Prasad C, B. Feuer, C. Hegde, G. Ramakrishnan, M. Goldblum, and C. White. When do neural nets outperform boosted trees on tabular data? In *Adv. Neural Inf. Process. Syst. (NeurIPS), Datasets and Benchmarks Track*, 2023. arXiv:2305.02997.

S. Müller, N. Hollmann, S. Pineda Arango, J. Grabocka, and F. Hutter. Transformers can do Bayesian inference. In *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022. arXiv:2112.10510.

C. Nadeau and Y. Bengio. Inference for the generalization error. *Mach. Learn.*, 52(3):239–281, 2003. doi: 10.1023/A:1024068626366.

NVIDIA. Kumo Tabular: Model card. Hugging Face model repository, `https://huggingface.co/nvidia/Kumo-Tabular`, 2026a. Accessed: Nov. 1, 2026.

NVIDIA. structured-data-models, version 0.2.1. Software, `https://github.com/NVIDIA/structured-data-models`, 2026b. Accessed: Nov. 1, 2026.

A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, et al. PyTorch: An imperative style, high-performance deep learning library. In *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2019. arXiv:1912.01703.

F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, et al. Scikit-learn: Machine learning in Python. *J. Mach. Learn. Res.*, 12:2825–2830, 2011.

L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin. CatBoost: Unbiased boosting with categorical features. In *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2018. arXiv:1706.09516.

J. Qu, D. Holzmüller, G. Varoquaux, and M. Le Morvan. TabICL: A tabular foundation model for in-context learning on large data. In *Proc. Int. Conf. Mach. Learn. (ICML)*, 2025. arXiv:2502.05564.

J. Qu, V. Hudovernik, M. Jurkovic, C. Lorenz, A. Nitta, D. Gordeev, F. Lopez, R. Bendias, G. Titericz Jr, A. S. Sokolovski, I. Hulseman, J. Leskovec, and M. Fey. NVIDIA Kumo Tabular sets a new accuracy-efficiency frontier for tabular prediction. Hugging Face community article, `https://huggingface.co/blog/nvidia/kumo-tabular`, September 2026. Accessed: Nov. 1, 2026.

R. Shwartz-Ziv and A. Armon. Tabular data: Deep learning is not all you need. *Inf. Fusion*, 81:84–90, 2022. doi: 10.1016/j.inffus.2021.11.011.

W. N. Street, W. H. Wolberg, and O. L. Mangasarian. Nuclear feature extraction for breast tumor diagnosis. In *Proc. SPIE 1905, Biomedical Image Processing and Biomedical Visualization*, pp. 861–870, 1993. doi: 10.1117/12.148698.

J. Vanschoren, J. N. van Rijn, B. Bischl, and L. Torgo. OpenML: Networked science in machine learning. *ACM SIGKDD Explorations Newsletter*, 15(2):49–60, 2014. doi: 10.1145/2641190.2641198.

A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, et al. Attention is all you need. In *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017. arXiv:1706.03762.

F. Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bull.*, 1(6):80–83, 1945. doi: 10.2307/3001968.

A Per-Dataset Results of the Main Benchmark

Tables 6 and 7 give the primary metric for every model and dataset (mean over 10 folds). All per-fold results, including the secondary metrics and timings, are in the supplementary material.

B Case Studies: Breast Cancer and Diabetes

The case studies use five stratified (classification) or random (regression) 80/20 splits with seeds 42–46, logistic or linear regression, random forests (500 trees) and histogram gradient boosting with defaults, and Kumo Tabular with the same settings as the main benchmark. Malignant tumours are the positive class. Tables 8 and 9 report means and standard deviations over the splits; Table 10 the within-dataset paired comparison with the strongest baseline; Tables 11 and 12 the training-size study (seeds 42–44); and Figure 4 the training-size curves and Kumo Tabular's quantile intervals. Because five resampled splits are dependent, these analyses are descriptive; no within-dataset difference is significant after the overlap correction and Holm's adjustment. A complete re-run of the same code reproduced all metrics exactly, and execution of

Table 6: Per-dataset log loss (classification), mean over 10 folds. Best value per dataset in bold.

Dataset	Kumo Tabular	Logistic reg.	Random forest	Extra trees	HistGB	XGBoost	LightGBM	CatBoost
balance-scale	**0.024**	0.278	0.535	2.853	0.407	0.319	0.331	0.255
banknote-authentication	**0.000**	0.048	0.027	0.014	0.015	0.015	0.023	0.008
blood-transfusion-service-center	**0.468**	0.477	0.782	2.490	0.579	0.609	0.559	0.490
climate-model-simulation-crashes	0.131	**0.131**	0.190	0.183	0.182	0.149	0.198	0.140
diabetes	**0.466**	0.485	0.480	0.483	0.632	0.721	0.628	0.480
kc2	**0.374**	0.386	0.754	0.821	0.515	0.531	0.526	0.384
mfeat-morphological	**0.559**	0.643	0.944	4.116	1.143	0.887	1.126	0.648
mfeat-zernike	**0.213**	0.374	0.687	3.743	0.967	0.724	0.938	0.541
pc1	**0.160**	0.227	0.181	0.545	0.297	0.240	0.315	0.180
pc3	**0.236**	0.283	0.265	0.332	0.357	0.353	0.355	0.258
pc4	**0.157**	0.229	0.197	0.224	0.236	0.223	0.226	0.180
qsar-biodeg	**0.284**	0.334	0.341	0.368	0.381	0.376	0.409	0.305
steel-plates-fault	**0.383**	0.773	0.577	0.573	0.669	0.568	0.647	0.506
vehicle	**0.199**	0.472	0.498	0.517	0.697	0.579	0.613	0.475
wdbc	**0.062**	0.078	0.112	0.108	0.110	0.091	0.119	0.082

Table 7: Per-dataset RMSE (regression), mean over 10 folds. Best value per dataset in bold.

Dataset	Kumo Tabular	Linear reg.	Random forest	Extra trees	HistGB	XGBoost	LightGBM	CatBoost
QSAR_fish_toxicity	**0.841**	0.954	0.884	0.882	0.897	0.944	0.899	0.857
airfoil_self_noise	**0.780**	4.823	1.648	1.390	1.789	1.462	1.790	1.466
cars	**2037.702**	2944.899	2307.586	2418.453	2481.138	2389.355	2484.833	2111.137
concrete_compressive_strength	**3.490**	10.436	4.777	4.509	4.376	4.148	4.406	3.928
energy_efficiency	0.416	2.943	0.449	0.438	0.482	**0.289**	0.482	0.291
red_wine	0.570	0.651	0.559	**0.543**	0.586	0.595	0.587	0.573

the same notebooks in a separate cloud environment (NVIDIA T4, PyTorch 2.11, float16) reproduced the results of Kumo Tabular, logistic/linear regression and random forests at the reported precision; histogram gradient boosting differed slightly there (for example, accuracy 0.960 instead of 0.967).

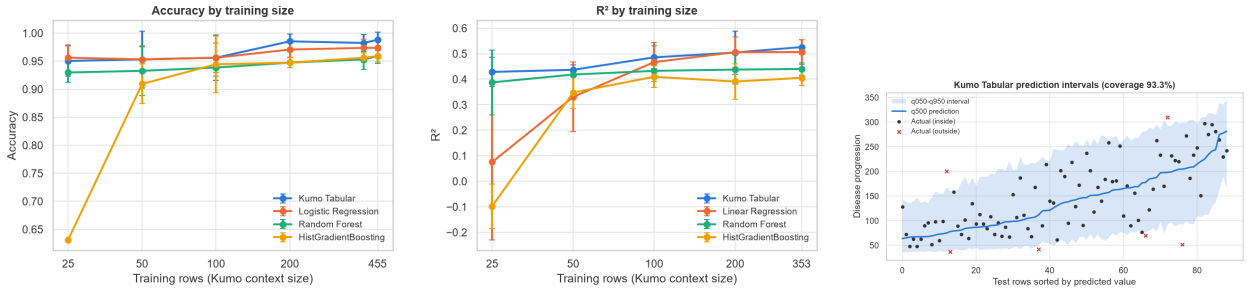


Figure 4: Case studies: classification accuracy and regression R^2 by number of training rows (mean ± one standard deviation over three seeds; logarithmic horizontal axis), and Kumo Tabular's median prediction with its nominal 90% interval on one diabetes split (coverage 93.3% on that split).

C Software and Hardware

Python 3.12.16, PyTorch 2.14.1 (Paszke et al., 2019) (CUDA 12.6), structured-data-models 0.2.1, scikit-learn 1.9.1, XGBoost 3.4.1, LightGBM 4.7.0, CatBoost 1.2.10, NumPy 2.5.4, pandas 3.1.0; NVIDIA GeForce GTX 980M (8 GB, compute capability 5.2), Intel Core i7-6700HQ, 16 GB RAM, Windows 10. Kumo Tabular weights: Hugging Face repository `nvidia/Kumo-Tabular`, revision `v1.0.1`, `small` checkpoints.

Table 8: Breast cancer classification results (mean ± standard deviation over five stratified splits, 114 test rows each). Malignant is the positive class. Time is fit plus prediction time per split. Best mean per metric in bold.

Model	Accuracy	Precision	Recall	F1	ROC-AUC	Time (s)
Kumo Tabular	**0.989 ± 0.010**	**0.995 ± 0.010**	**0.976 ± 0.029**	**0.985 ± 0.014**	**0.998 ± 0.003**	1.099 ± 0.012
Logistic Reg.	0.977 ± 0.012	0.990 ± 0.013	0.948 ± 0.031	0.968 ± 0.017	0.995 ± 0.004	0.019 ± 0.001
Random Forest	0.954 ± 0.013	0.947 ± 0.031	0.929 ± 0.024	0.938 ± 0.018	0.988 ± 0.011	1.204 ± 0.063
HistGB	0.967 ± 0.016	0.976 ± 0.024	0.933 ± 0.043	0.953 ± 0.022	0.994 ± 0.007	0.274 ± 0.020

Table 9: Diabetes regression results (mean ± standard deviation over five random splits, 89 test rows each). Kumo Tabular uses the median quantile $q_{0.5}$ as point prediction. Best mean per metric in bold.

Model	MAE	RMSE	R^2	Time (s)
Kumo Tabular	**42.48 ± 3.10**	**52.88 ± 3.47**	**0.503 ± 0.039**	0.627 ± 0.053
Linear Reg.	43.75 ± 3.59	53.27 ± 4.03	0.496 ± 0.046	0.003 ± 0.000
Random Forest	45.23 ± 2.63	56.16 ± 3.31	0.440 ± 0.022	0.833 ± 0.024
HistGB	45.85 ± 0.72	58.52 ± 1.67	0.390 ± 0.042	0.190 ± 0.031

Table 10: Paired comparison of Kumo Tabular with the strongest baseline over the five splits. $\overline{d}$ is the mean advantage of Kumo (positive favours Kumo for every metric), s_d its standard deviation. The corrected test and interval use the Nadeau–Bengio variance correction for overlapping training sets; Holm adjustment is applied within each task. No result is significant at $\alpha = 0.05$ after adjustment.

Task	Metric	W/T/L	$\overline{d}$	s_d	Corrected 95% CI	p_{naive}	p_{corr}	$p_{\text{corr}}^{\text{Holm}}$	p_{Wilcoxon}
Classification	Accuracy	5/0/0	0.0123	0.0078	[−0.0023, 0.0269]	0.025	0.080	0.313	0.062 (n=5)
	Precision	2/3/0	0.0051	0.0111	[−0.0156, 0.0258]	0.362	0.531	0.535	0.500 (n=2)
	Recall	5/0/0	0.0286	0.0106	[0.0087, 0.0484]	0.004	0.016	0.081	0.062 (n=5)
	F1	5/0/0	0.0172	0.0109	[−0.0031, 0.0376]	0.024	0.078	0.313	0.062 (n=5)
	ROC-AUC	4/1/0	0.0030	0.0034	[−0.0034, 0.0094]	0.126	0.267	0.535	0.125 (n=4)
Regression	MAE	4/0/1	1.27	1.10	[−0.78, 3.32]	0.061	0.161	0.484	0.125 (n=5)
	RMSE	3/0/2	0.39	1.43	[−2.28, 3.07]	0.573	0.704	1.000	0.625 (n=5)
	R^2	3/0/2	0.007	0.028	[−0.046, 0.060]	0.603	0.727	1.000	0.625 (n=5)

Table 11: Classification accuracy by training-set size (mean ± standard deviation over three seeds; full test split).

n	Kumo Tabular	Logistic Reg.	Random Forest	HistGB
25	0.950 ±0.027	**0.956** ±0.023	0.930 ±0.018	0.632 ±0.000
50	**0.953** ±0.051	**0.953** ±0.022	0.933 ±0.044	0.909 ±0.035
100	**0.956** ±0.040	**0.956** ±0.026	0.939 ±0.015	0.944 ±0.051
200	**0.985** ±0.013	0.971 ±0.013	0.947 ±0.009	0.947 ±0.009
400	**0.982** ±0.015	0.974 ±0.015	0.953 ±0.018	0.956 ±0.009
455	**0.988** ±0.013	0.974 ±0.015	0.959 ±0.013	0.959 ±0.010

Table 12: Regression R^2 by training-set size (mean ± standard deviation over three seeds; full test split).

n	Kumo Tabular	Linear Reg.	Random Forest	HistGB
25	**0.429** ±0.058	0.075 ±0.304	0.388 ±0.127	−0.097 ±0.087
50	**0.437** ±0.019	0.331 ±0.137	0.418 ±0.008	0.348 ±0.063
100	**0.486** ±0.045	0.467 ±0.076	0.433 ±0.024	0.410 ±0.043
200	0.505 ±0.085	**0.506** ±0.060	0.438 ±0.057	0.391 ±0.069
353	**0.526** ±0.028	0.507 ±0.048	0.440 ±0.024	0.406 ±0.030